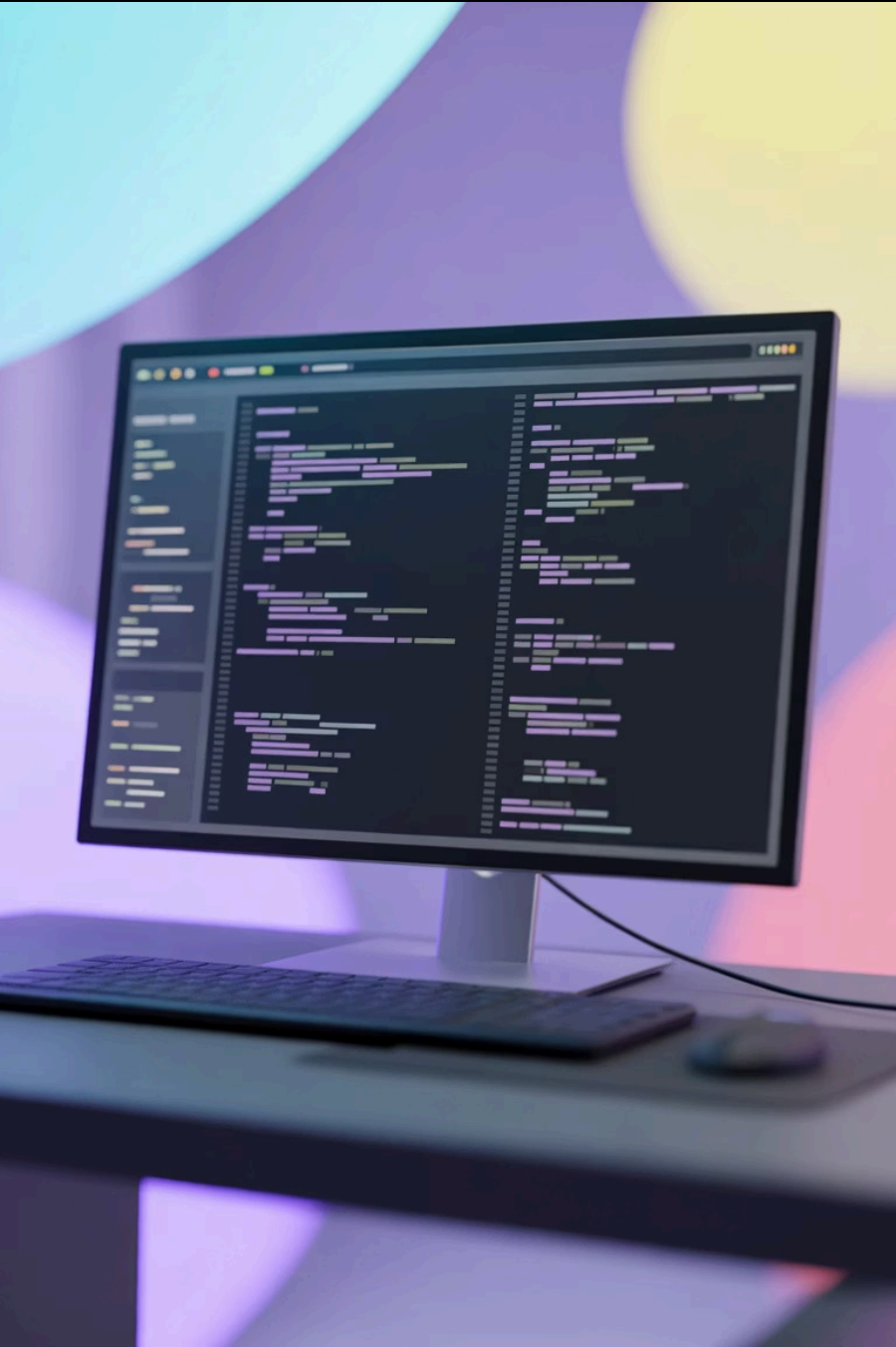




Testes de Contrato e Integração

A robustez da migração para microserviços depende de testes abrangentes. Testes de contrato evitam regressões inesperadas entre o consumidor (monólito) e o provedor (novo microserviço), enquanto testes de integração garantem que o provedor responda exatamente conforme o contrato especificado.

O resultado esperado é um conjunto de testes de contrato (validação da OpenAPI) e testes de integração (via ferramentas como Supertest) "verdes", assegurando a conformidade e a estabilidade da API.

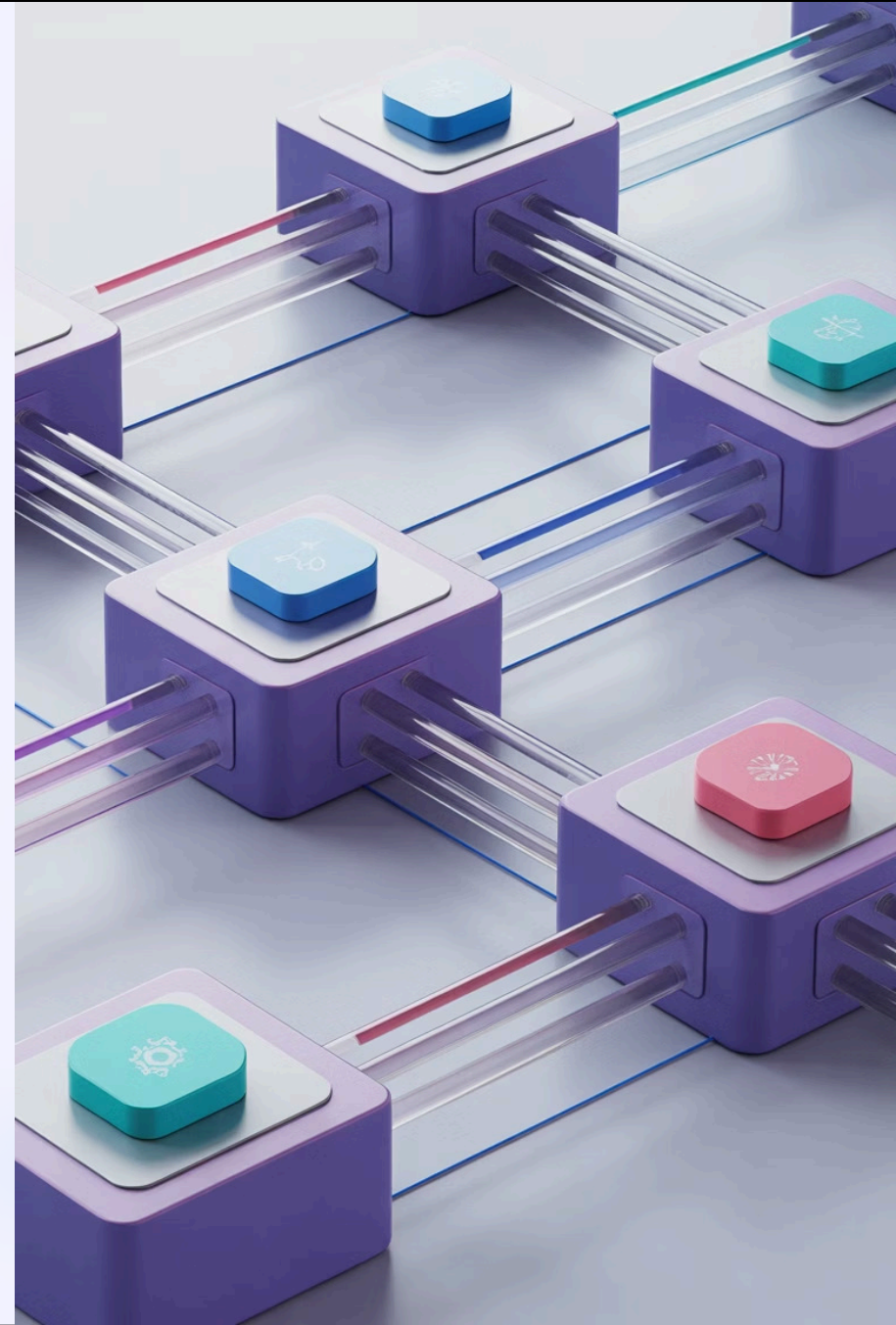
Testes de Contrato: Fundamentos

O Que São

A garantia da integridade e consistência de uma API é fundamental em arquiteturas de microserviços. Os testes de contrato verificam se a implementação do serviço adere estritamente à sua especificação OpenAPI.

Por Que Importam

O princípio fundamental é validar a presença e a estrutura de elementos esperados no arquivo openapi.yaml de cada pacote. Isso evita que alterações acidentais ou intencionais na especificação passem despercebidas.



Microservices Architecture Testure Testing

Contracteract

uitegation cralloq

hheatiin Opensopi
ococnatiine

Viaidche o'irecchardó
specicicating

SpertzAPI

SuperTest

SuperTest

Implementando Testes de Contrato

01

Criar Arquivo de Teste

Local: packages/users-service/test/contract/openapi.spec.ts

02

Configurar Ferramentas

Usar Jest com setup TypeScript já presente no repositório

03

Adicionar Dependências

Incluir yaml como devDependency no users-service

04

Implementar Validações

Verificar paths, schemas e estruturas essenciais

Requisitos dos Testes de Contrato

Leitura do OpenAPI

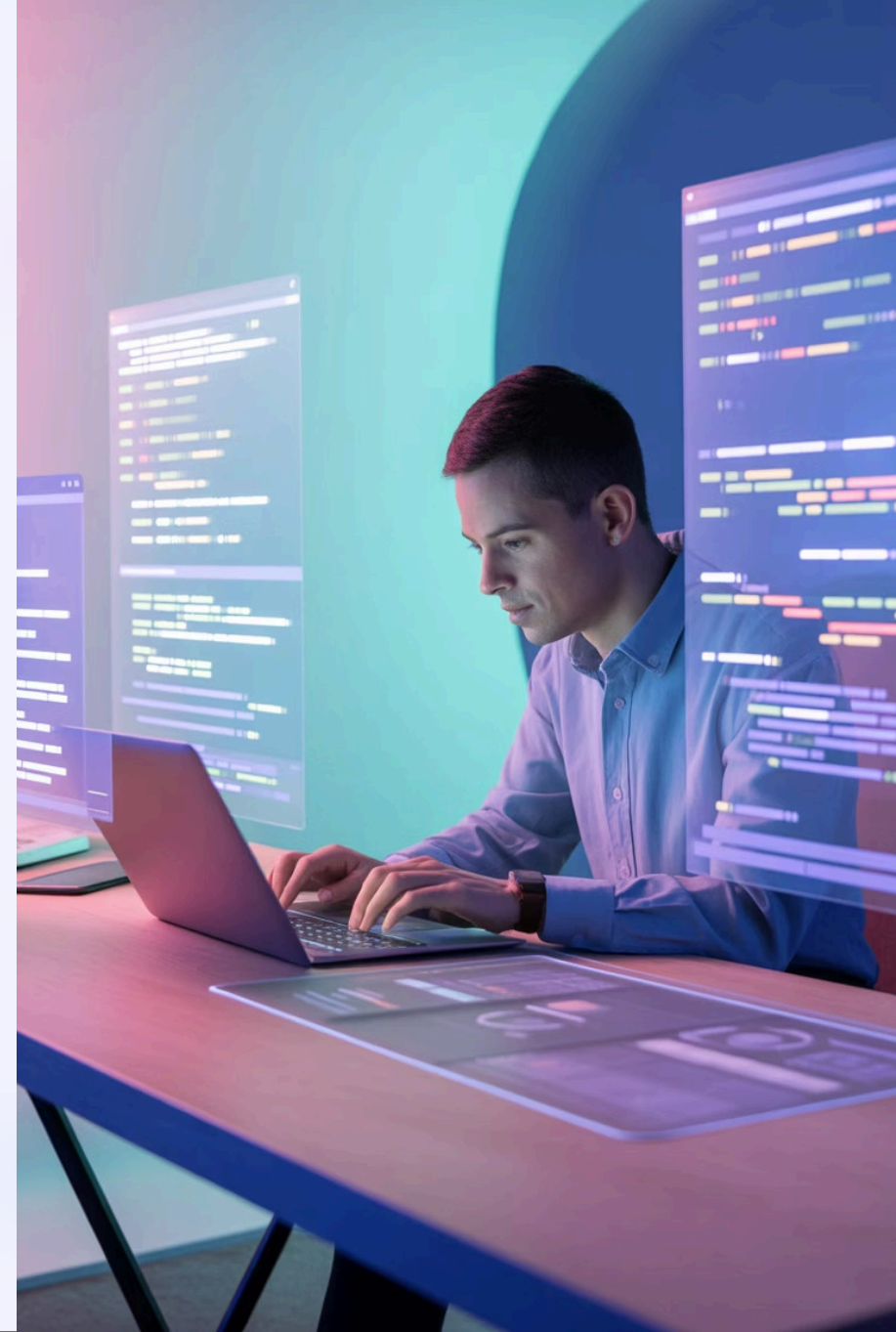
O teste deve ler o arquivo `openapi.yaml` e fazer o parse com `yaml`, transformando em objeto `JavaScript`.

Asserções Obrigatórias

- `spec.paths['/users']` está definido
- `spec.paths['/users'].post` está definido
- `spec.components.schemas.CreateUser` está definido
- `spec.components.schemas.User` está definido

Validação Adicional

Verificar que a referência do `requestBody` do `POST /users` aponta para `#/components/schemas/CreateUser` ou que o schema de request contenha os campos esperados.



Validação OpenAPI na Prática

A implementação prática para validar a especificação OpenAPI de um microserviço envolve duas etapas principais:



Leitura do openapi.yaml

O arquivo de especificação é lido e parseado para um objeto JavaScript/TypeScript utilizando a biblioteca yaml.



Validação de Elementos

Asserções garantem que endpoints (paths) e modelos de dados (schemas) essenciais estejam definidos na especificação.

Exemplo de Código: Leitura e Parse

```
import fs from 'fs';  
import YAML from 'yaml';  
  
// Lê e parseia o arquivo openapi.yaml do serviço  
const raw = fs.readFileSync('packages/users-service/openapi.yaml', 'utf8');  
const spec = YAML.parse(raw);
```

```
// Valida a presença de paths e schemas específicos  
expect(spec.paths['/users']).toBeDefined();  
expect(spec.paths['/users'].post).toBeDefined();  
expect(spec.components.schemas.CreateUser).toBeDefined();  
expect(spec.components.schemas.User).toBeDefined();
```

Este código exemplifica a base para um teste de contrato robusto, garantindo que o serviço de usuários mantenha sua integridade e que seu contrato seja sempre válido e completo.

Como Executar os Testes

A partir da Raiz do Monorepo

Recomendado: Esta é a forma mais comum, permitindo gerenciar e rodar testes de múltiplos pacotes a partir de um único ponto.

```
npx jest packages/users-  
service/test/contract/openapi.spec.ts --runInBand
```

A partir do Diretório do Pacote

Útil se você estiver focando exclusivamente no desenvolvimento e teste de um único serviço.

```
cd packages/users-service  
npx jest test/contract/openapi.spec.ts --runInBand
```

O uso do comando `--runInBand` é recomendado para garantir que os testes sejam executados sequencialmente, evitando possíveis conflitos de recurso ou estado.

Gerenciando Dependências de Testes



Quando Instalar

Se encontrar erros como "No tests found" ou falhas na transformação de arquivos TypeScript, pode ser necessário instalar dependências localmente no serviço específico.



Dependências Necessárias

Jest, ts-jest, @types/jest e yaml devem ser instalados como devDependencies no diretório do serviço.



Configuração Inicial

Use `npx ts-jest config:init` para gerar uma configuração mínima do Jest, permitindo personalizações futuras.

```
cd packages/users-service  
npm install --save-dev jest ts-jest @types/jest yaml  
npx ts-jest config:init
```

Alternativa Simples: Validação sem TypeScript

Para cenários de depuração ágil ou quando a validação da presença de chaves específicas em seu arquivo YAML é urgente, uma solução simplificada está disponível via script Node.js direto.

```
node -e "const fs=require('fs'); const YAML=require('yaml'); const spec=YAML.parse(fs.readFileSync('packages/users-service/openapi.yaml','utf8')); console.log(Object.keys(spec.paths | | {}));"
```

Este comando executa um script que lê, parseia e exibe as chaves de paths do seu arquivo OpenAPI, servindo como uma ferramenta de diagnóstico rápido para verificar a integridade da sua especificação.





Interpretando Resultados do Jest

✓ Sucesso

Quando todos os testes são aprovados, o Jest exibirá a mensagem `PASS` para o arquivo de teste executado, acompanhada de um resumo conciso. Isso valida que a especificação OpenAPI está em conformidade com o contrato esperado.

✗ Falha

Em caso de falha, a saída do Jest será detalhada, apontando exatamente qual asserção não foi atendida. Você pode ver erros indicando a ausência de um path específico ou a falta de definição de um schema crucial.

Otimização e Melhores Práticas



Alternativa JavaScript Puro

Crie uma versão em JavaScript puro do teste (`openapi.spec.js`) usando `require('yaml')` e `fs` diretamente, oferecendo flexibilidade para equipes que operam predominantemente em JavaScript.



Integração Contínua (CI)

Adicione um job à sua Pipeline de CI que execute os testes de contrato. Configure para falhar o pipeline caso os testes não sejam aprovados, detectando regressões precocemente.





OpenAPI Generator: Visão Geral

O OpenAPI Generator é uma ferramenta poderosa que automatiza a criação de artefatos de código a partir de uma especificação OpenAPI. Ele desempenha um papel crucial na manutenção da consistência entre a documentação da API e sua implementação, além de acelerar significativamente o ciclo de desenvolvimento.

A ferramenta lê uma especificação OpenAPI e gera automaticamente clientes de API, servidores stubs, SDKs e documentação, garantindo a consistência entre a especificação e o código.

Capacidades do OpenAPI Generator

Geração de Cliente

Permite criar uma biblioteca completa para o consumidor da API. Isso inclui métodos tipados, modelos de dados e lógica de serialização/deserialização, facilitando a integração de novas aplicações.

Geração de Server Stub

Cria um esqueleto de projeto de servidor, com rotas, modelos e controladores vazios, permitindo que os desenvolvedores foquem na implementação da lógica de negócio sem se preocupar com a estrutura inicial da API.



Comandos Básicos do Generator

A utilização do OpenAPI Generator não só economiza tempo, mas também minimiza erros manuais, assegurando que todas as partes envolvidas no desenvolvimento de uma API trabalhem com uma base comum e atualizada.

1

Gerar Client TypeScript

```
npx @openapitools/openapi-generator-cli generate \  
-i packages/users-service/openapi.yaml \  
-g typescript-axios \  
-o generated/users-client
```

2

Gerar Server Stub Node.js

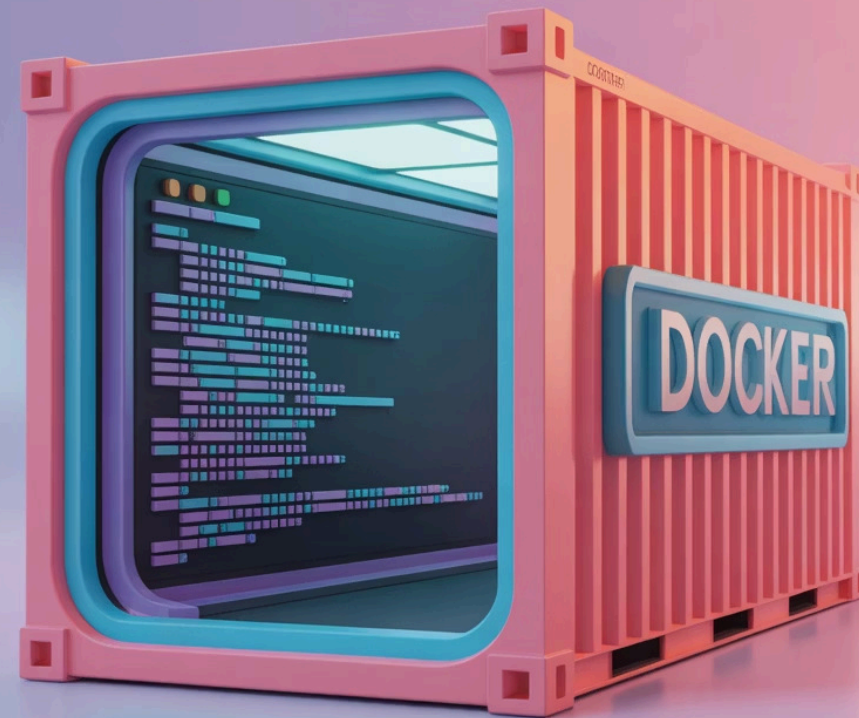
```
npx @openapitools/openapi-generator-cli generate \  
-i packages/users-service/openapi.yaml \  
-g nodejs-express-server \  
-o generated/users-server
```

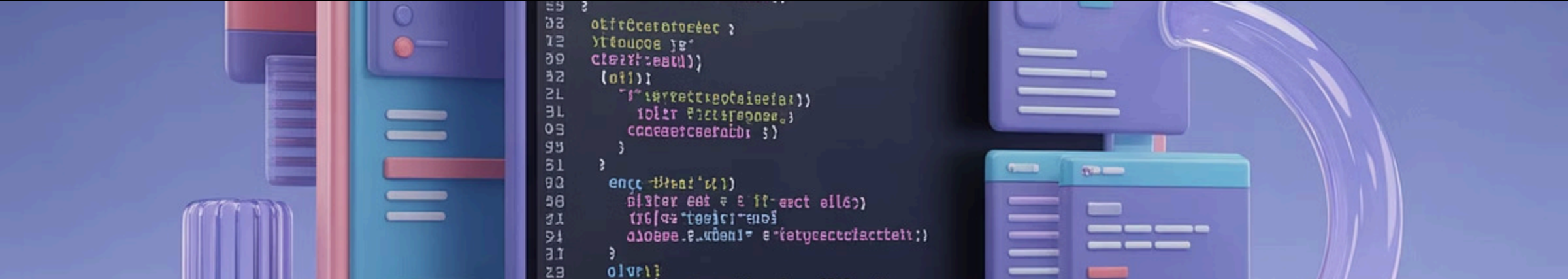
Execução via Docker

Para desenvolvedores que preferem não instalar a CLI do OpenAPI Generator globalmente ou que trabalham em ambientes containerizados, o Docker oferece uma maneira elegante de executar a ferramenta.

```
docker run --rm -v ${PWD}:/local openapitools/openapi-generator-cli  
generate \  
-i /local/packages/users-service/openapi.yaml \  
-g typescript-axios \  
-o /local/generated/users-client
```

Este comando executa o gerador dentro de um contêiner Docker, mapeando o diretório de trabalho atual para /local dentro do contêiner. Isso permite que o gerador acesse o arquivo openapi.yaml e grave os artefatos gerados diretamente no seu sistema de arquivos local.





Conteúdo dos Artefatos Gerados

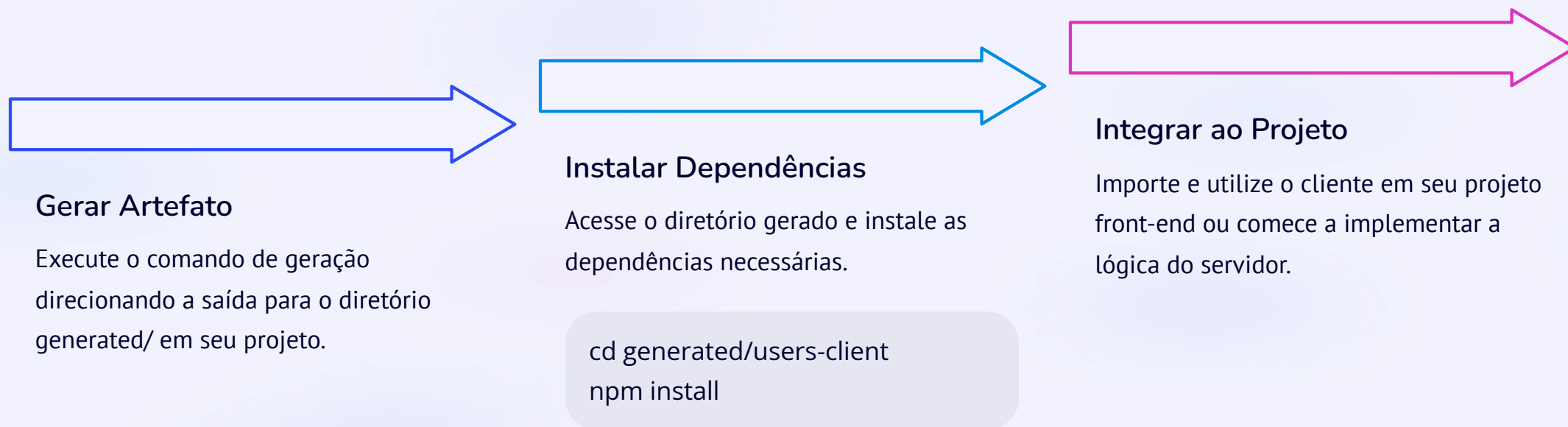
Client (typescript-axios)

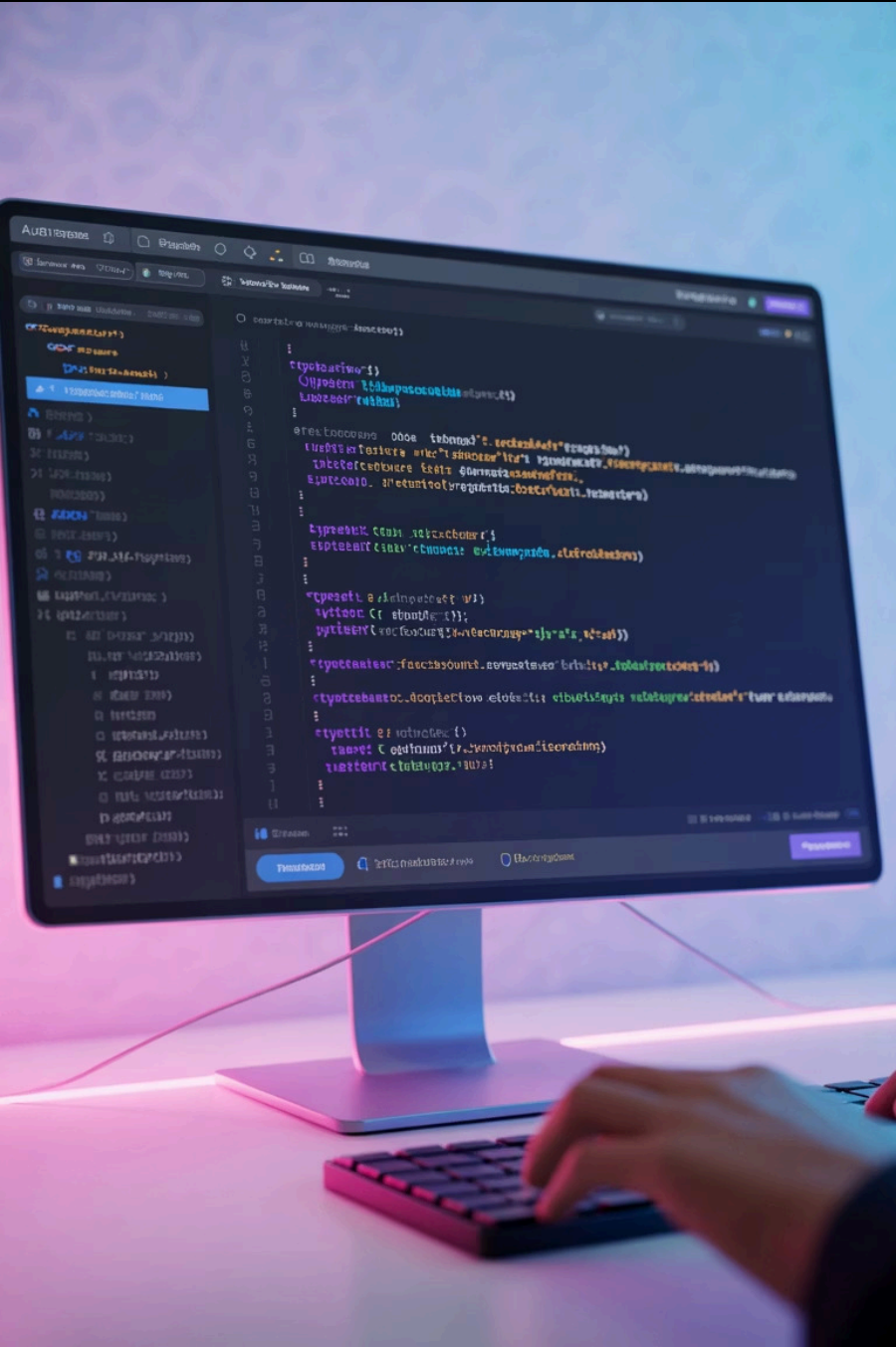
- Modelos de dados (interfaces/classes) tipados para requisição e resposta
- Serviços/APIs com funções para cada endpoint
- README detalhado com instruções de uso
- Lógica de serialização e desserialização implementada

Server Stub (nodejs-express-server)

- Estrutura de rotas baseada na especificação OpenAPI
- Esqueletos de controladores (métodos vazios)
- Modelos de dados representando os schemas da API
- Instruções básicas para iniciar o servidor

Guia de Uso Rápido





Consumindo a API com Client Gerado

O client gerado para TypeScript, utilizando Axios, oferece uma interface tipada e intuitiva para interagir com a API. Você pode chamar os endpoints da sua API como métodos de uma classe, com autocompletar e validação de tipos.

```
import { UsersApi } from './generated/users-client';
```

```
const api = new UsersApi({ basePath: 'http://localhost:3010' });  
await api.createUser({ email: 'a@b.com', name: 'Aluno' });
```

O client cuida de todos os detalhes de requisição HTTP, serialização e desserialização, permitindo que você se concentre na lógica de negócio.

Implementando no Server Stub

01

Abrir Diretório

Navegue até `generated/users-server` após a geração do stub.

02

Instalar Dependências

Execute `npm install` para configurar o ambiente do servidor.

03

Implementar Controladores

Preencha os métodos vazios com a lógica que responderá às requisições dos endpoints.

04

Rodar Servidor

Inicie o servidor conforme as instruções no `README.md` gerado (geralmente `npm start`).



Boas Práticas com OpenAPI Generator

Versionamento

Versione tanto a especificação OpenAPI (openapi.yaml) quanto a versão exata do OpenAPI Generator utilizada para garantir reprodutibilidade.

Gerenciamento de Código

Decida se o código gerado será commitado. Em projetos reais, é comum regenerar o código em um pipeline de CI/CD.

Revisão e Adaptação

Sempre revise o código gerado antes de usá-lo em produção. Ajustes são normalmente necessários para alinhar aos padrões do projeto.

Compatibilidade OpenAPI

Esteja ciente de que alguns geradores podem ter melhor suporte para OpenAPI 3.0 do que para 3.1. Verifique a compatibilidade.